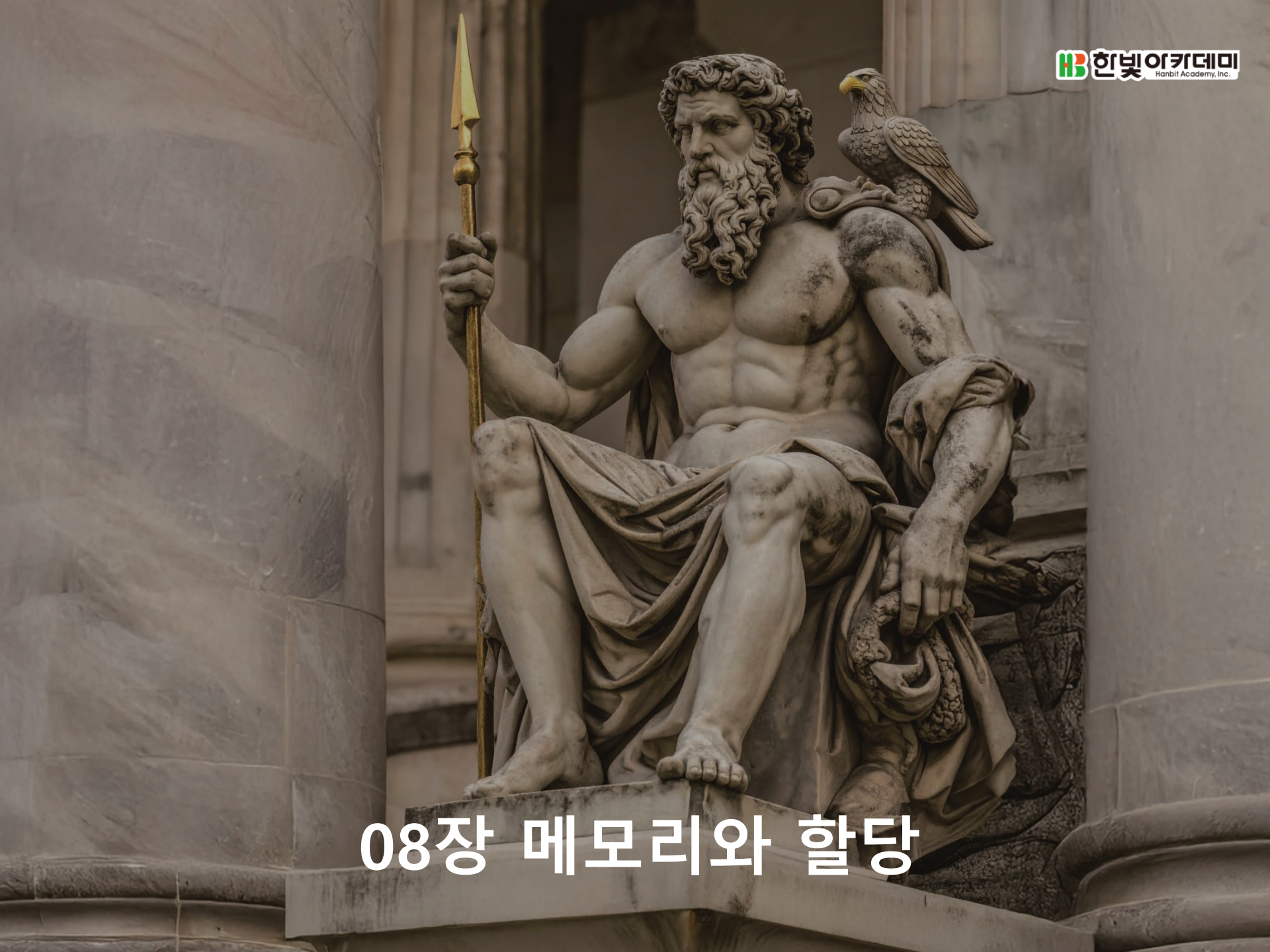




08장 메모리와 할당

학습목표

- 메모리의 구성과 역할, 메모리를 보호하는 방식과 컴파일 과정을 살펴본다.
- 논리 주소와 물리 주소의 차이점과 주소를 변환하는 이유를 이해한다.
- 주요 메모리 할당 방식과 단편화 해결 기법을 비교하여 학습한다.

목차

1. 메모리 구조와 프로그램 실행 준비
2. 프로세스의 메모리 배치
3. 메모리 할당 기법
4. [심화] 버디 시스템

01

메모리 구조와
프로그램 실행 준비

01 메모리 구조와 프로그램 실행 준비

1. 메모리와 프로그램 실행 구조

◆ 메모리의 기본 구조

- 폰노이만 구조에서 메모리는 CPU의 유일한 작업 공간.
- 모든 프로그램은 메모리에 올라와야 실행가능.



그림 8-1 개념적으로 표현한 메모리

01 메모리 구조와 프로그램 실행 준비

1. 메모리와 프로그램 실행 구조

◆ 커널의 메모리 배치

- 운영체제의 보안 향상
- 사용자 프로세스의 메모리 할당에 유리



(a) 커널 영역을 낮은 주소에 배치



(b) 커널 영역을 높은 주소에 배치

커널을 높은 주소에 배치하면
사용자 영역으로부터 멀어진다.



그림 8-2 커널 영역과 사용자 영역의 분리

01 메모리 구조와 프로그램 실행 준비

1. 메모리와 프로그램 실행 구조

◆메모리 보호 방식

- 프로세스가 허용되지 않은 메모리 영역에 접근하지 못하도록 제한하는 기능.
- 하드웨어와 운영체제가 협력하여 수행.

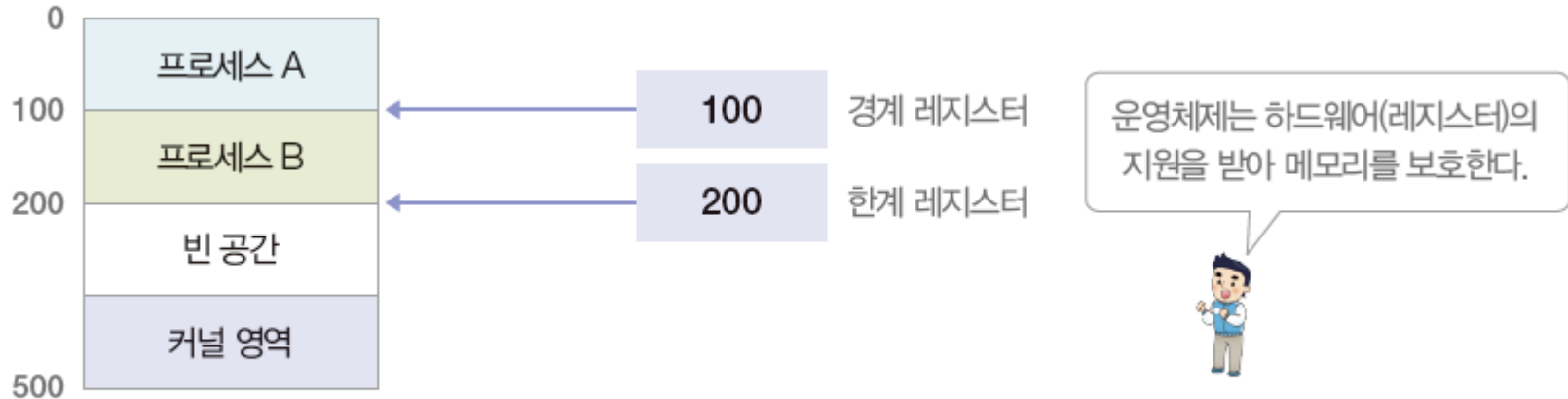


그림 8-3 메모리 보호

01 메모리 구조와 프로그램 실행 준비

2. 컴파일과 실행 준비

◆컴파일러의 작업 이해하기

- 고급 언어로 작성된 코드를 컴퓨터가 이해할 수 있는 기계어로 변환하는 것.
- 컴파일러, 인터프리터

◆컴파일의 주요 단계

- 소스 코드 분석
- 최적화 및 목적 코드 생성
- 연결 단계

01 메모리 구조와 프로그램 실행 준비

2. 컴파일과 실행 준비

◆ 컴파일의 주요 단계

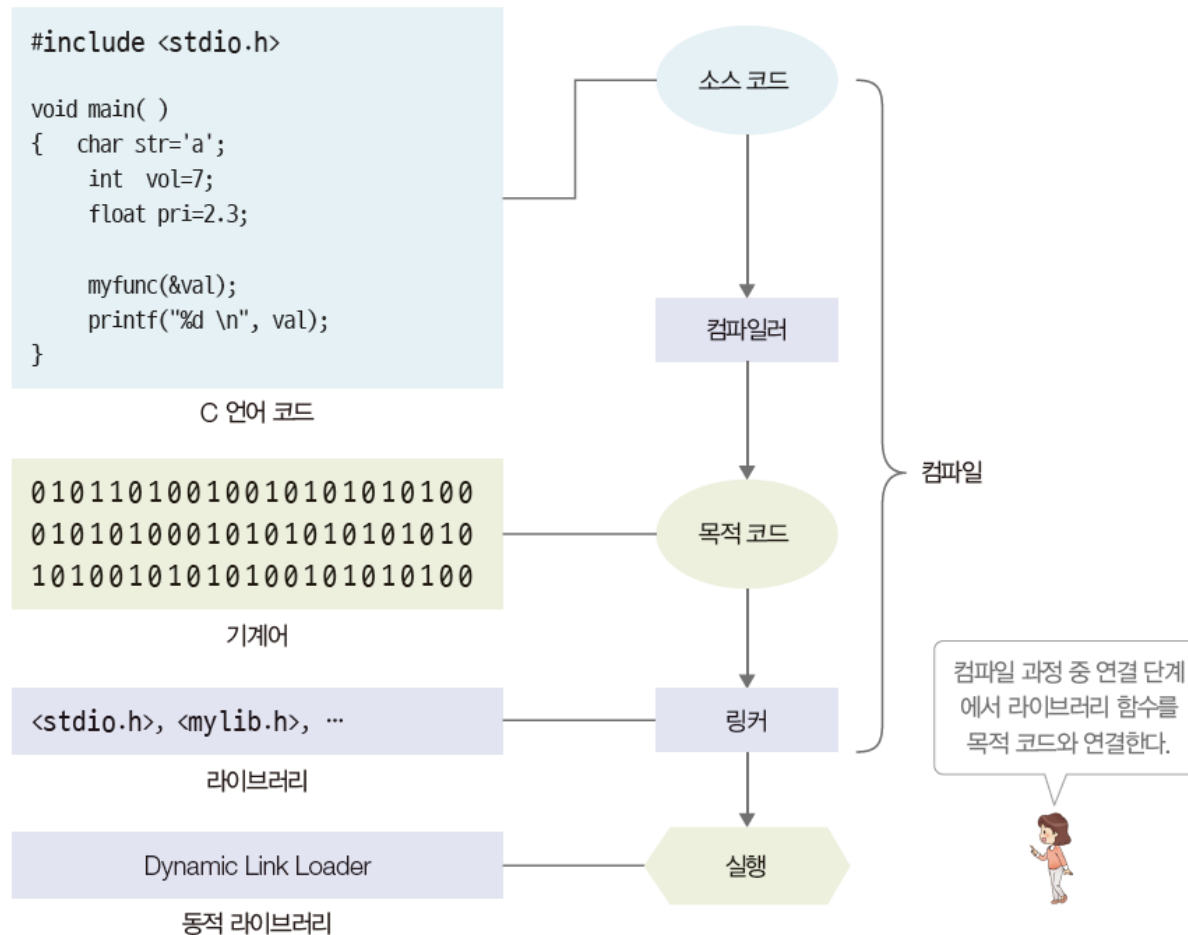


그림 8-4 컴파일 과정

01 메모리 구조와 프로그램 실행 준비

2. 컴파일과 실행 준비

◆ 컴파일의 주요 단계

- 목적코드에서 라이브러리 함수를 연결단계에서 삽입하는 이유
 - 함수의 실제 코드를 한 번만 삽입.
 - 함수의 기계어 코드를 한 번만 연결.
 - 여러 사람이 공동 작업하기 편리.

01 메모리 구조와 프로그램 실행 준비

2. 컴파일과 실행 준비

◆ 분할 컴파일과 링킹

- 분할 컴파일: 여러 개의 소스 파일을 하나의 실행 파일로 만드는 것
 - 다중 소스 코드

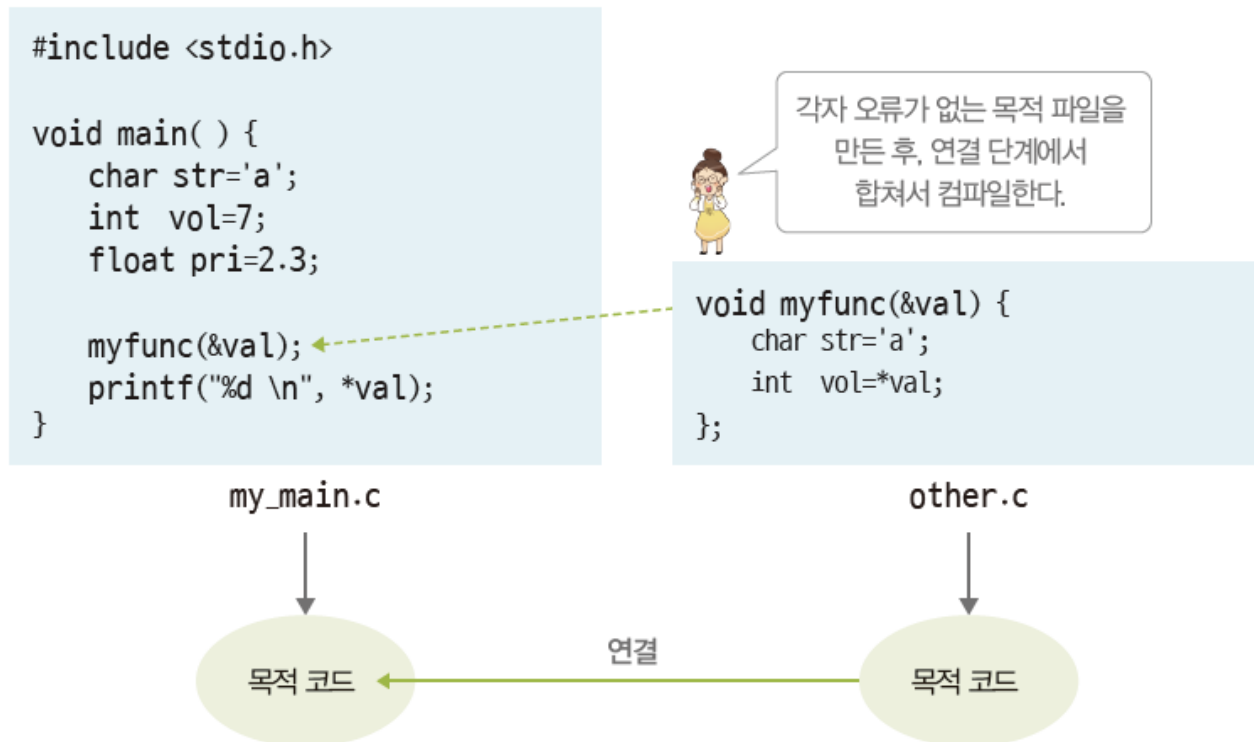


그림 8-5 다중 소스 코드 작업에서의 연결 단계

01 메모리 구조와 프로그램 실행 준비

2. 컴파일과 실행 준비

◆ 동적 라이브러리

- 실행 파일을 만들 때 함수의 자리를 비워두는 대신
실행 시점에서 라이브러리의 해당 함수를 불러와 실행.
- 라이브러리 함수의 기계어 코드가 변경되어도 새로 컴파일할 필요가 없음.
- DLL(Dynamic Link Loader)

02

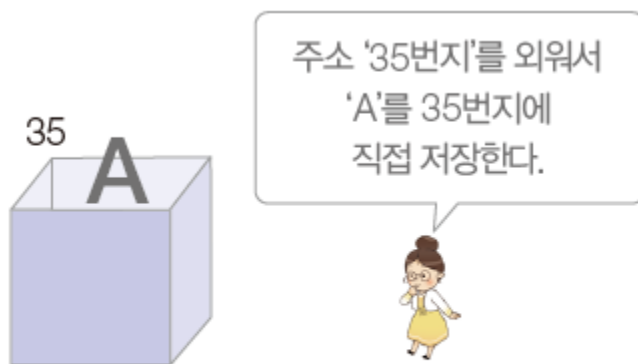
프로세스의 메모리 배치

02 프로세스의 메모리 배치

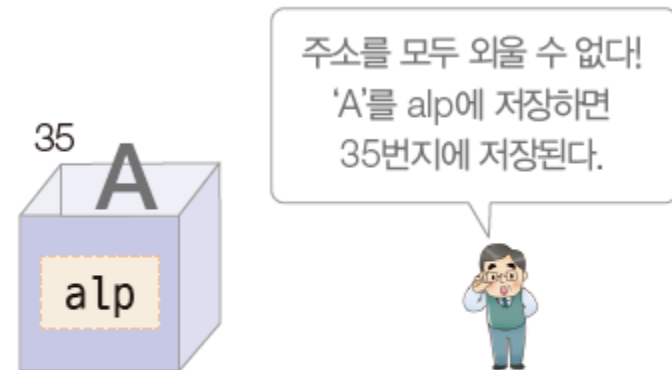
1. 메모리 주소

◆ 변수와 주소 추상화

- 변수
 - 저장된 주소에 이름을 붙인 것
 - 메모리 주소를 문자로 추상화한 것



(a) 주소로 메모리에 접근



(b) 추상화된 이름으로 메모리에 접근

그림 8-6 메모리 주소의 추상화

02 프로세스의 메모리 배치

1. 메모리 주소

◆ 논리 주소와 물리 주소

- 논리 주소: 프로세스 입장에서의 메모리 주소
- 물리 주소(실제 주소): 메모리 입장에서의 물리 메모리 주소

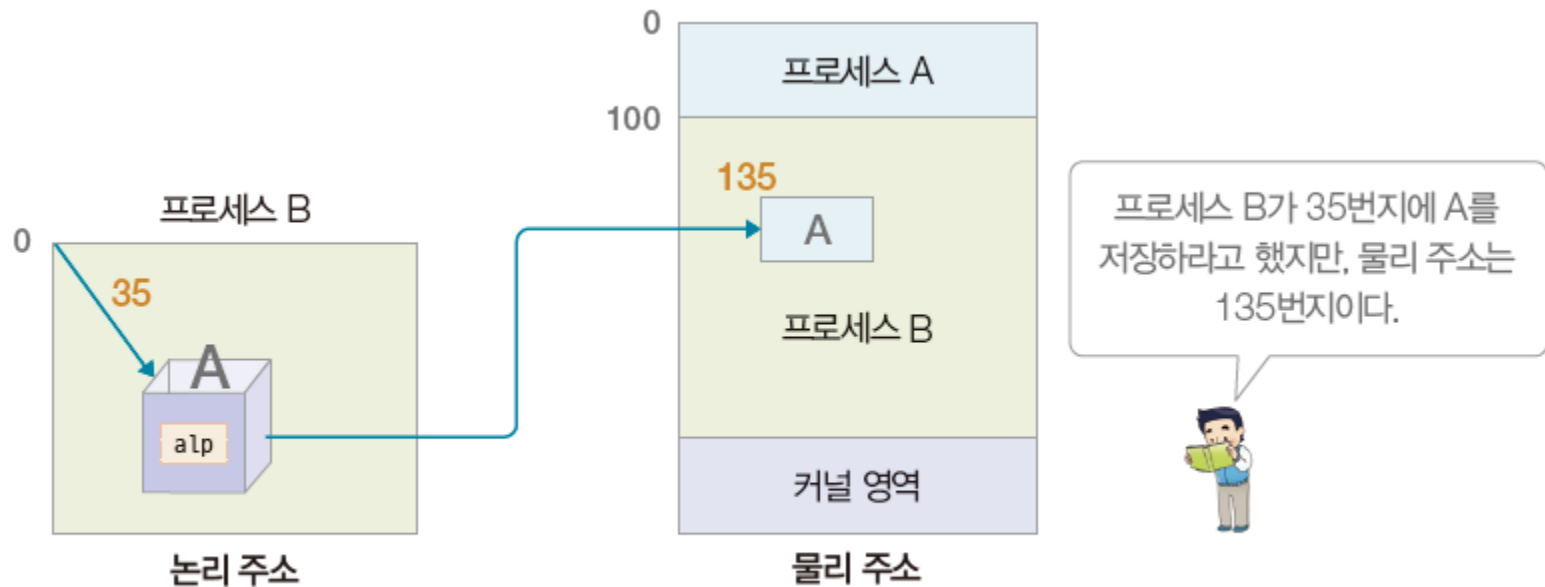


그림 8-7 논리 주소와 물리 주소

02 프로세스의 메모리 배치

1. 메모리 주소

◆ 논리 주소와 물리 주소

- 메모리 관리 유닛이 논리 주소 → 물리 주소의 변환 작업을 수행

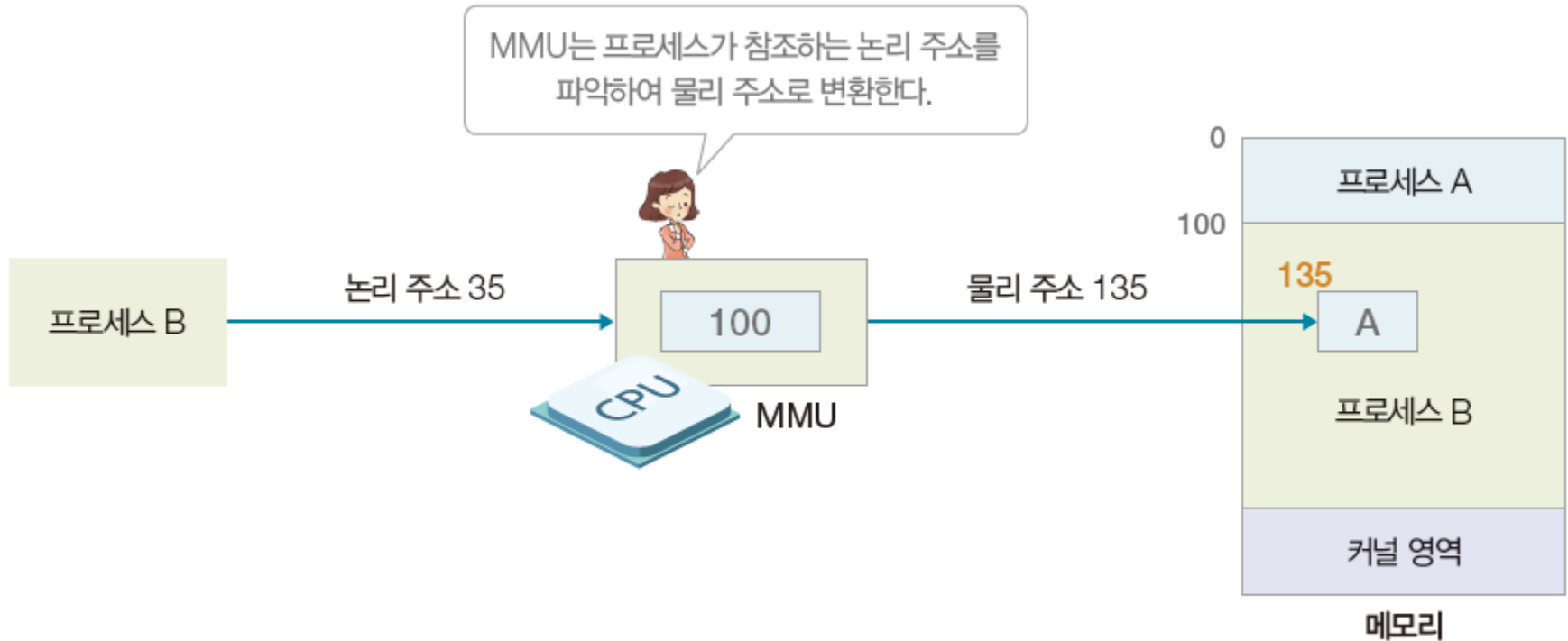


그림 8-8 논리 주소를 물리 주소로 변환

02 프로세스의 메모리 배치

1. 메모리 주소

◆ 32비트 CPU와 64비트 CPU의 메모리 주소

- CPU의 비트 수 = 한 번에 처리하는 데이터의 최대 크기
 - CPU가 접근할 수 있는 메모리 주소의 범위와 밀접한 관계 있음

02 프로세스의 메모리 배치

2. 메모리 오버레이

◆ 메모리 오버레이의 개념

- 고깃덩이 - 프로그램
- 도마 - 물리 메모리

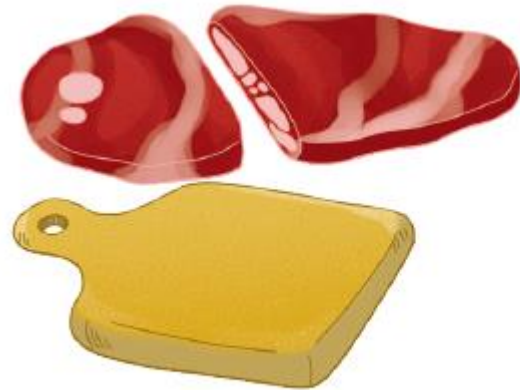
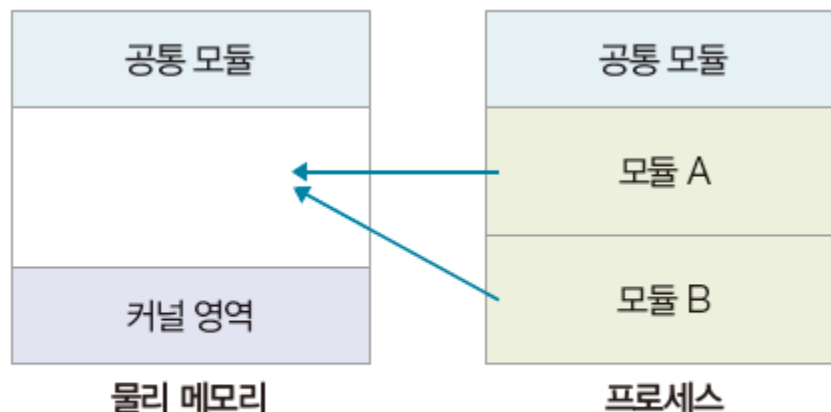


그림 8-9 메모리보다 큰 프로그램을 실행하는 방법



프로그램의 일부만 메모리에 올라와 있어도 실행할 수 있다.



그림 8-10 메모리 오버레이

02 프로세스의 메모리 배치

2. 메모리 오버레이

◆메모리 오버레이의 개념

- 프로그램의 일부만 메모리에 존재해도 작업 수행 가능.
- 현대 컴퓨터 시스템은 가상 메모리 기술 사용.
- 운영체제가 하드디스크와 메모리를 연동하여
전체 프로그램이 한꺼번에 메모리에 올라오지 않아도 실행될 수 있도록 지원.

02 프로세스의 메모리 배치

3. 스왑

- 사용하지 않는 데이터를 일시적으로 저장장치로 옮겨 메모리 공간을 확보하는 기법.
- 데이터를 내보내는 작업을 스왑 아웃이라고 함.



그림 8-11 보관 창고의 임시 저장 공간

02 프로세스의 메모리 배치

3. 스왑

- 스왑 영역: 저장장치에서 데이터를 보관하는 장소.

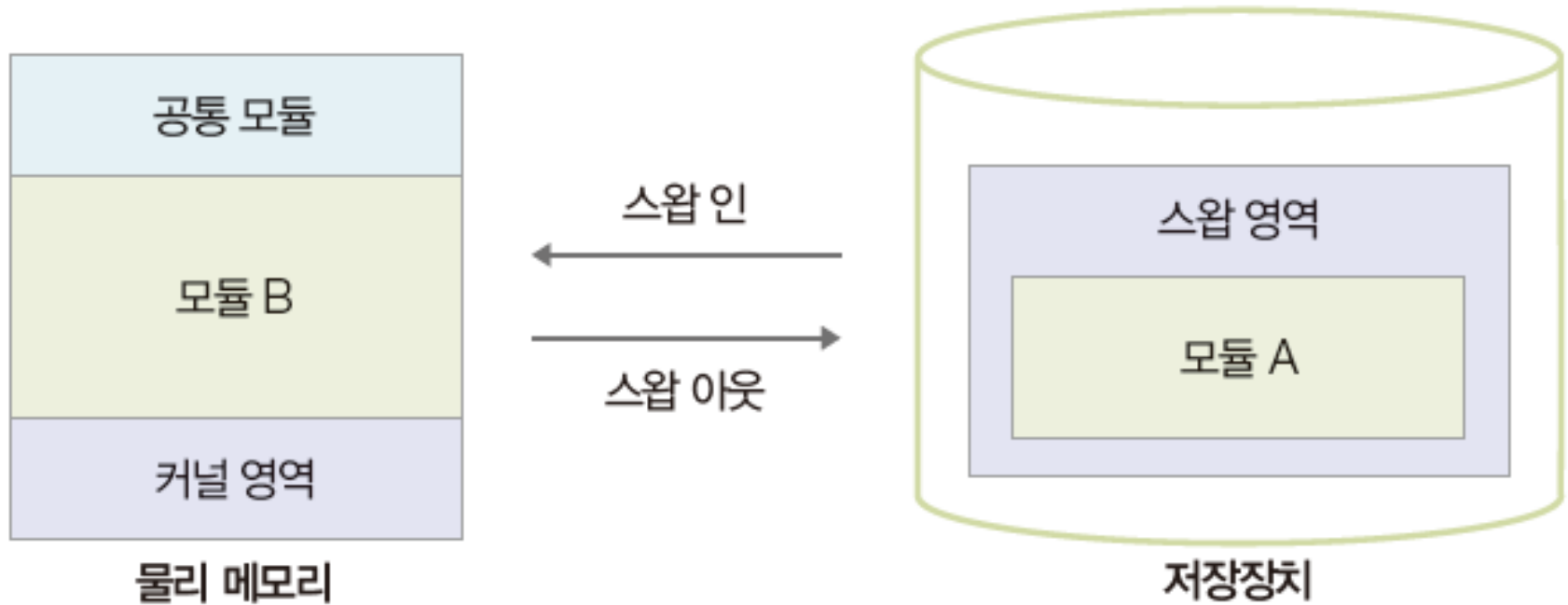


그림 8-12 스왑 인과 스왑 아웃

03

메모리 할당 기법

03 메모리 할당 기법

1. 가변 분할과 고정 분할

◆ 메모리의 분할

- 가변 분할: 프로세스의 크기에 따라 메모리를 나누는 방식.
- 고정 분할: 프로세스의 크기와 관계없이 메모리를 같은 크기로 나누는 방식.



(a) 가변 분할



(b) 고정 분할

그림 8-13 가변 분할과 고정 분할

03 메모리 할당 기법

1. 가변 분할과 고정 분할

◆가변 분할과 고정 분할의 구현

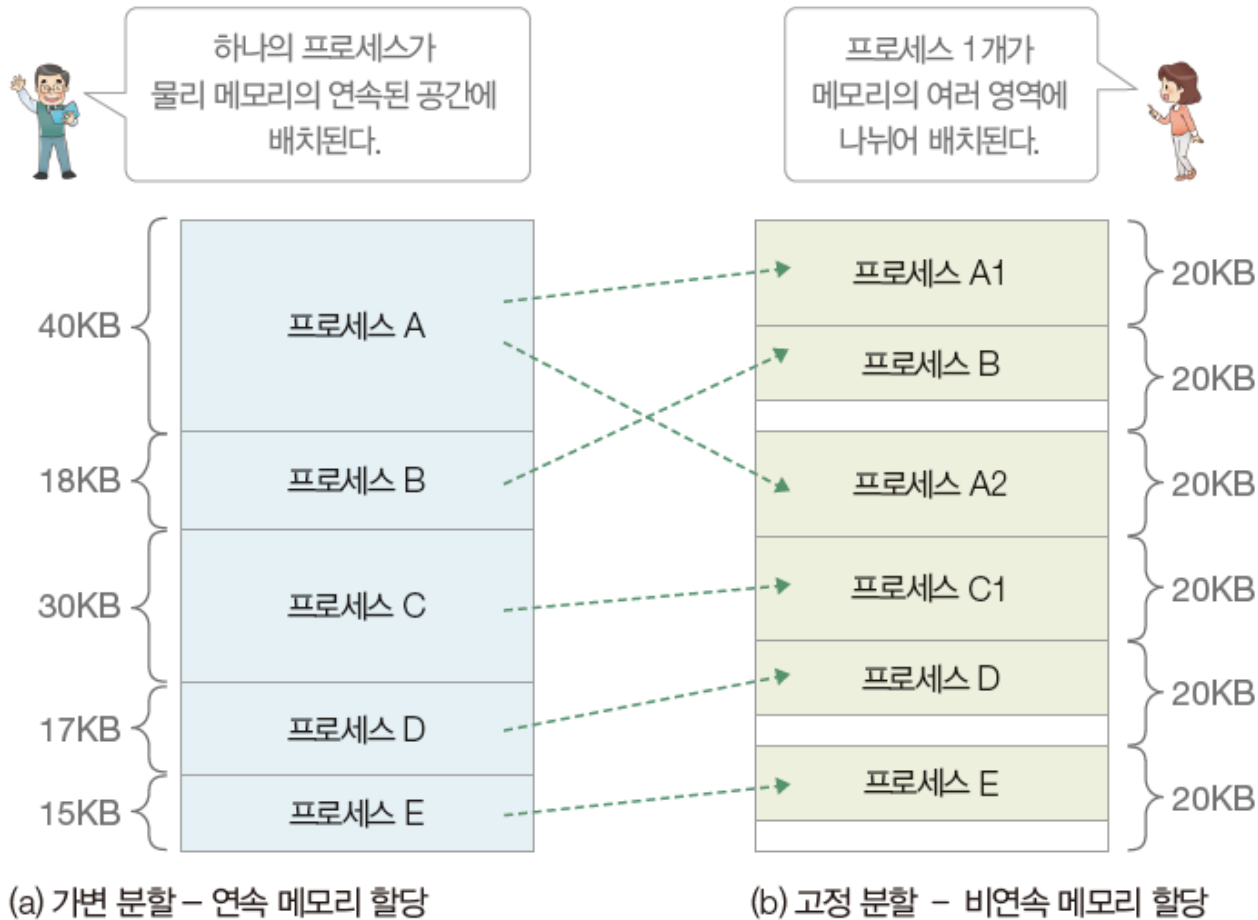


그림 8-14 메모리 분할 방식의 구현

03 메모리 할당 기법

2. 가변 분할의 메모리 관리

◆ 외부 단편화

- 단편화: 메모리 외부에 조각이 생기는 현상
- 외부 단편화: 가변 분할에서 프로세스의 바깥쪽에 단편화 조각이 생김.

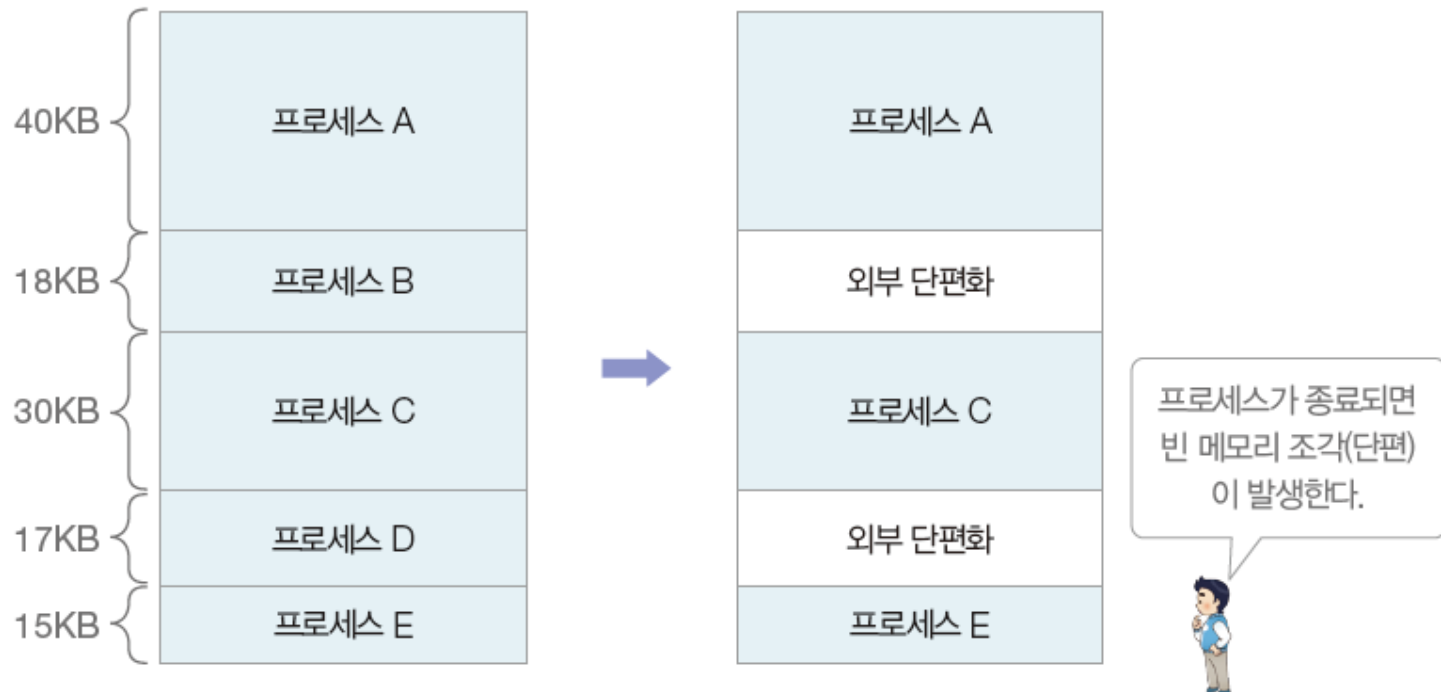


그림 8-15 가변 분할과 외부 단편화

03 메모리 할당 기법

2. 가변 분할의 메모리 관리

◆메모리 배치 전략

- 작은 조각이 발생하지 않도록 프로세스를 배치하는 작업.
- 가변분할에서의 배치 전략
 - 최초 배치: 단편화를 고려하지 않는 방식.
 - 최적 배치: 메모리의 빈 공간을 모두 확인한 후 프로세스와 크기가 가장 가까운 빈 공간에 프로세스를 배치하는 방식.
 - 최악 배치: 최적 배치와 정반대
 - 버디 시스템

03 메모리 할당 기법

2. 가변 분할의 메모리 관리

◆ 메모리 배치 전략

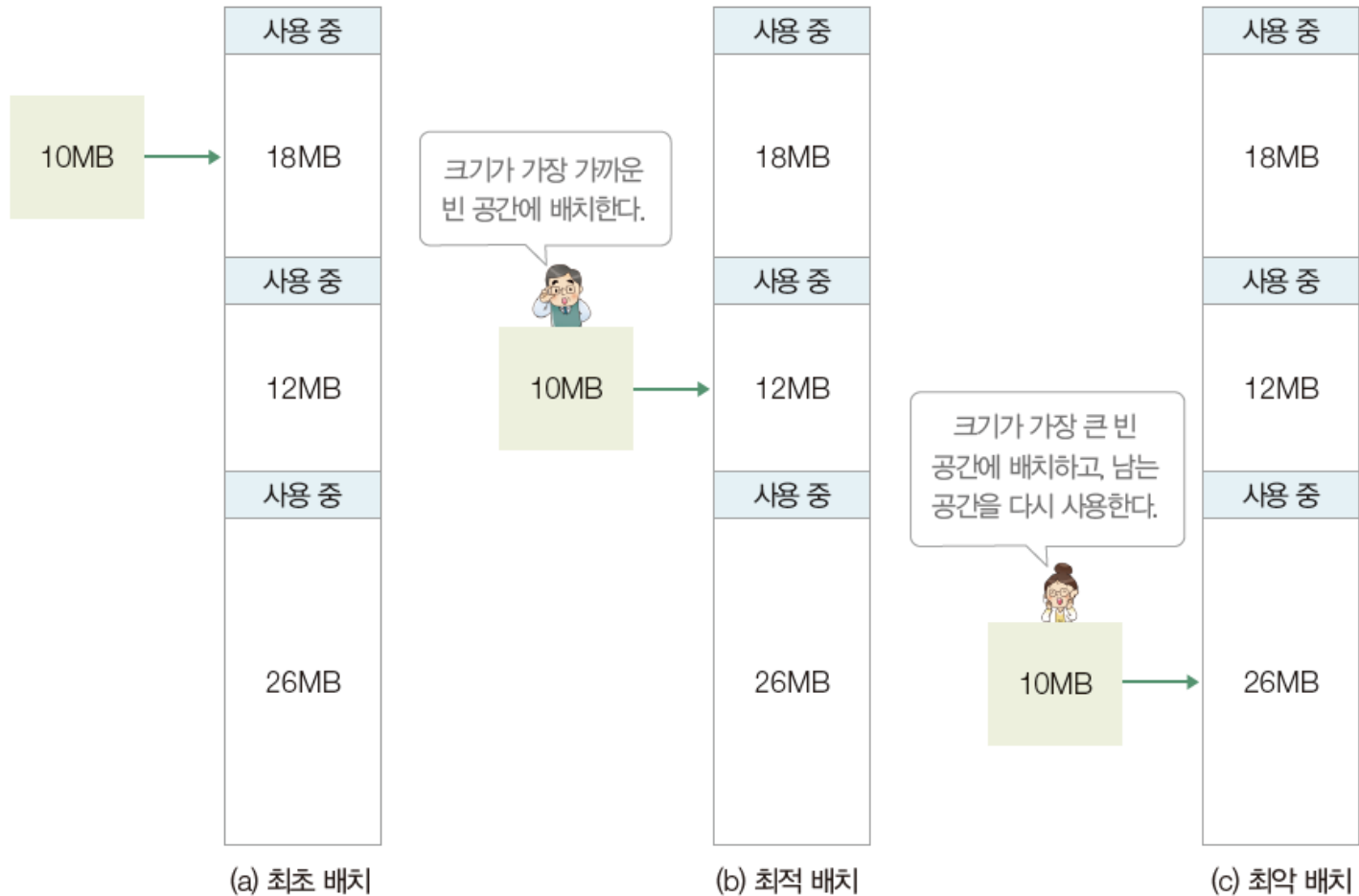


그림 8-16 메모리 배치 전략 비교

03 메모리 할당 기법

2. 가변 분할의 메모리 관리

◆ 조각 모음

- 서로 떨어져 있는 여러 개의 빈 공간을 합치는 작업.

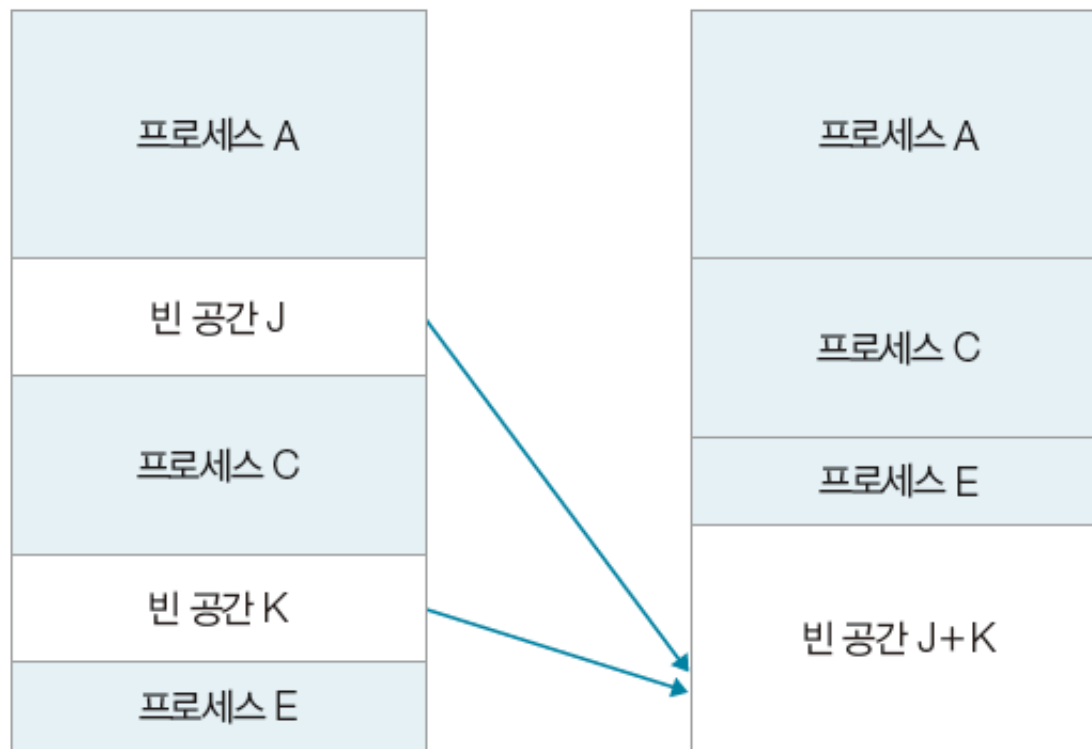


그림 8-17 조각 모음

03 메모리 할당 기법

3. 고정 분할의 메모리 관리

- 고정 분할: 프로세스의 크기에 상관없이 메모리를 같은 크기로 나눔.
- 내부 단편화: 메모리 안쪽에 공간이 남는 현상



그림 8-18 고정 분할의 프로세스 배치

04

[심화] 버디 시스템

04 [심화] 버디 시스템

◆ 버디 시스템의 작동 방식

- 가변 분할과 고정 분할의 중간 구조

- ① 프로세스가 들어가는 최소 크기가 될 때까지 전체 메모리를 1/2로 자르기를 반복
- ② 나뉜 메모리의 각 구역에는 프로세스가 1개만 들어감
- ③ 프로세스가 종료되면 주변의 빈 조각을 합쳐서 하나의 큰 덩어리를 만듦

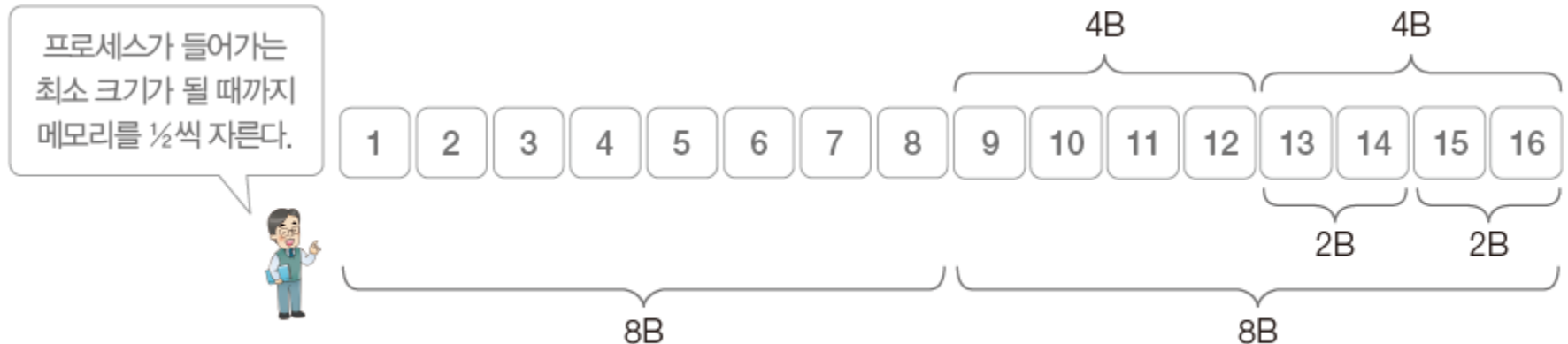


그림 8-19 버디 시스템의 메모리 분할

04 [심화] 버디 시스템

◆ 버디 시스템의 작동 방식

프로세스 도착 순서 : A(3B) - B(2B) - C(2B) - D(1B) - E(4B) - F(1B) - G(2B)

- ① 오른쪽 8B를 4B, 2B, 2B로 나누어 A(3B), B(2B), C(2B)를 배정



- ② 왼쪽 8B를 4B, 2B, 1B, 1B로 나누어 D(1B)를 배정



- ③ 왼쪽 4B에 E(4B), D 옆 1B에 F(1B), 남은 2B에 G(2B)를 배정



그림 8-20 프로세스가 여러 개일 때의 버디 시스템

04 [심화] 버디 시스템

◆ 버디 시스템의 단편화 완화

- 버디 시스템은 가변 분할과 고정 분할의 특징을 모두 가짐.
- 메모리를 프로세스 크기대로 나눔.
- 하나의 구역에 다른 프로세스가 들어갈 수 없어 단편화가 발생.
- 비슷한 크기의 빈 조각이 서로 모여 있어 가변 분할보다 공간 관리가 효과적.

Thank You!